

GitHub Actions CI Assessment Submission Evidence

1. GitHub Repository

****Repository URL:****

<https://github.com/Tanusha-tyagi/github-skills-challenge>

The repository contains the Python application, tests, and GitHub Actions workflows.

2. Pull Request Used for Validation

****Pull Request URL:****

<https://github.com/DebbieAUG/github-skills-challenge/pull/160>

****Pull Request Title:****

Add Python test and coverage workflows

The pull request contains the Python test workflow, Python coverage workflow, and additional tests required to meet the coverage threshold.

3. Development Environment

A Python virtual environment was created and activated in the GitHub Codespace.

The following dependencies were installed:

- Project dependencies from `requirements.txt`
- `pytest`

- `coverage`
- `pytest-cov`

The application source code and original test files were not modified to manipulate the assessment results.

4. Python Test Workflow

****Workflow file:****

`python-test.yml`

The workflow:

- Runs on pull requests targeting the `main` branch.
- Does not use a `push` trigger.
- Runs on `ubuntu-latest`.
- Checks out the repository.
- Sets up Python 3.11.
- Installs project dependencies.
- Installs pytest and coverage tools.
- Executes the test suite using detailed output:

```
``bash
python -m pytest -vv
``
```

5. Python Coverage Workflow

****Workflow file:****

``python-coverage.yml``

The workflow:

- Runs on pull requests targeting the `main` branch.
- Uses `ubuntu-latest`.
- Checks out the repository.
- Sets up Python 3.11.
- Installs project dependencies and testing tools.
- Generates coverage for the `src` directory.
- Publishes coverage information to the pull request.
- Uses pull-request write permission.
- Enforces a minimum coverage requirement of 90%.

Coverage commands:

```
``bash
python -m pytest -vv --cov=src --cov-report=term-missing
coverage report --fail-under=90
...
```

6. Test Results

The complete test suite was executed successfully.

****Final test result:****

- Total tests: 14
- Passed: 14

- Failed: 0

Command used:

```
```bash
python -m pytest -vv --cov=src --cov-report=term-missing
```
```

7. Coverage Results

Coverage was generated for the `src` directory.

****Final coverage result:****

- Total coverage: 96%
- Required minimum coverage: 90%
- Coverage threshold: Passed

The command below completed successfully:

```
```bash
coverage report --fail-under=90
```
```

8. Additional Coverage Tests

The following file was added to cover previously untested application branches:

```
```text
```

tests/coverage\_additional\_test.py

...

The additional tests cover:

- Anomaly detection conditions
- Empty event handling
- Topic clearing
- Invalid negative calculation inputs
- Fibonacci calculation
- Pipeline data loading
- Pipeline command-line execution

No application logic was changed.

## ## 9. Final Repository Files

The final implementation contains:

```text

.github/workflows/python-test.yml

.github/workflows/python-coverage.yml

tests/coverage_additional_test.py

...

10. Commit History

The implementation was committed and pushed to the `main` branch.

| Commit | Description |
|-----------|---------------------------------------|
| --- | --- |
| `7960131` | Add pull request Python test workflow |
| `033277b` | Add pull request coverage workflow |
| `8b3f721` | Add tests to meet coverage threshold |

11. Screenshot Evidence

The following screenshots should be included with the submission:

1. GitHub Actions page showing both workflows.
2. Successful Python test workflow execution.
3. Detailed test output showing all tests passed.
4. Pull request coverage comment showing the coverage result.
5. Final repository file tree showing both workflow files.
6. Pull request page showing the submitted changes.

12. Validation Note

The workflows are configured and recognized by GitHub Actions. Local validation confirms that all 14 tests pass and that coverage is 96%, which is above the required 90% threshold.

For fork-based pull requests, GitHub may require selecting **Approve and run** before workflow jobs execute. Once approved, the GitHub-hosted workflow logs and pull-request coverage comment can be captured as final evidence.

Conclusion

The Python application now has continuous integration and automated test-coverage workflows. The test suite executes with detailed output, coverage is generated for the `src` directory, coverage results can be published to pull requests, and the workflow enforces a minimum 90% coverage requirement.
